



# HAFTA 1

## Dijkstra — En Kısa Yol

### Sıfırdan, Adım Adım Konu Anlatımı

PLAN-101 · Rota Planlamanın Temel Taşı

**Kime:** Berk

**Ön koşul:** Lise matematiği. Grafin *ne* olduğunu bilmene bile gerek yok — 1. sayfada anlatıyorum.

**İçerik:** 0. Bölüm: graf nedir, neden yol haritası bir graftır  
1. Bölüm: algoritma — sezgi, kural, elle tam çözüm  
2. Bölüm: neden doğru çalışır (kanıt) + negatif tuzağı  
3. Bölüm: kod — naif sürümden yığına, satır satır  
4. Bölüm: 10 örnek soru + tam çözümler  
5. Bölüm: BURST — `mission.py`'nin içi

**Tarih:** 29 Ağustos 2026

*Bu belgeyi bitirdiğinde BURST'ün rota planlayıcısını  
**satır satır** okuyabiliyor olacaksın —  
ve “neden düğüm değil de kenar üzerinde arıyoruz” sorusunu  
tahtada çizerek anlatabileceksin.*

# İçindekiler

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>0. BÖLÜM — Graf: Yol Haritasının Matematiği</b>                | <b>3</b>  |
| 1 Önce problem: araç nereye gideceğini nasıl biliyor?             | 3         |
| 2 Graf nedir? (Gerçekten sıfırdan)                                | 3         |
| 2.1 Ağırlık: kenarın bedeli . . . . .                             | 3         |
| 2.2 Yönlü graf: tek yönlü sokaklar . . . . .                      | 4         |
| 2.3 Grafi bilgisayara nasıl anlatırız: komşuluk listesi . . . . . | 4         |
| 2.4 Bizim gerçek grafımız . . . . .                               | 5         |
| <b>1. BÖLÜM — Algoritma</b>                                       | <b>6</b>  |
| 3 Problem tam olarak nedir?                                       | 6         |
| 4 Neden “hepsini dene” işe yaramaz                                | 6         |
| 5 Sezgi: su baskını                                               | 6         |
| 6 Algoritma — iki tanımla üç adım                                 | 7         |
| 7 ÖRNEK 1 — Elle tam çözüm (bunu mutlaka kâğıtta tekrarla)        | 8         |
| <b>2. BÖLÜM — Neden Doğru Çalışıyor?</b>                          | <b>12</b> |
| 8 Ana iddia ve kanıt                                              | 12        |
| 9 Negatif ağırlık: Dijkstra’nın tek ölümcül zaafı                 | 12        |
| <b>3. BÖLÜM — Kod</b>                                             | <b>14</b> |
| 10 Sürüm 1: en yalın hâli (yığın yok)                             | 14        |
| 11 Neden yavaş? Ve öncelik kuyruğu                                | 14        |
| 12 Sürüm 2: yığınlı — gerçek kullanacağın sürüm                   | 15        |
| 12.1 “Bayat kayıt” satırı — burayı anlamadan geçme . . . . .      | 15        |
| 13 Yolu geri kurmak                                               | 16        |
| 14 Ne kadar hızlı? BURST’ün sayılarıyla                           | 16        |
| <b>4. BÖLÜM — Örnek Sorular ve Tam Çözümler</b>                   | <b>17</b> |
| 15 Soru 1 — Isınma: elle tam tablo                                | 17        |
| 16 Soru 2 — Tek yön: ters şeride giremezsin                       | 17        |

|                                                         |    |
|---------------------------------------------------------|----|
| 17 Soru 3 — “Hedefi gördüm, durabilir miyim?”           | 18 |
| 18 Soru 4 — Beraberlik: iki yol da 5                    | 19 |
| 19 Soru 5 — Ulaşılamayan hedef                          | 19 |
| 20 Soru 6 — Negatif ağırlık: yanlış yakala              | 19 |
| 21 Soru 7 — Ağırlığı değiştir, cevap değişsin           | 20 |
| 22 Soru 8 — Tercihli bölge: tüneli kazandırmak          | 20 |
| 23 Soru 9 — Engel çıktı: rotayı yeniden hesapla         | 21 |
| 24 Soru 10 — Kavşak kuralı: düğüm araması neden yetmez? | 22 |
| 5. BÖLÜM — BURST: mission.py’nin İçi                    | 24 |
| 25 Rota planlayıcı yığının neresinde?                   | 24 |
| 26 En kritik karar: durum bir <i>kenardır</i>           | 24 |
| 27 Başlangıç sorunu: araç düğümün üstünde değil         | 24 |
| 28 Ana döngü — satır satır                              | 25 |
| 28.1 Dört filtre, dört gerçek dünya problemi . . . . .  | 26 |
| 29 Geri sarma: kenar zincirinden sürülebilir çizgiye    | 27 |
| 30 Özet: hangi ders kodun neresinde                     | 27 |
| 6. BÖLÜM — Sonrası                                      | 28 |
| 31 A <sup>*</sup> : Dijkstra + sezgi                    | 28 |
| 32 Tuzaklar — tek sayfa özet                            | 28 |
| 33 Görevler                                             | 29 |

# 0. BÖLÜM — Graf: Yol Haritasının Matematiği

## 1 Önce problem: araç nereye gideceğini nasıl biliyor?

BURST'ün aracı pistte duruyor. Panelde bir hedef seçtin: “3 numaralı park cebine git.” Araç şimdi ne yapacak?

Üç ayrı soru var ve karıştırılmamalı:

1. **Hangi yollardan gideyim?** — Sağa mı sapayım, düz mü devam edeyim, tüneli mi kullanayım? Bu **rota planlama** (global planlama). Cevabı bir *yol listesi*: “şu kavşaktan sağa, sonra düz, sonra soldaki park koridoru.”
2. **Şeridin neresinde olayım?** — Rota “düz devam” dedi, ama önümde engel var; 1.2 m sola kayıp geçmeliyim. Bu **yörünge planlama** (yerel planlama). Sende bunu `trajectory.py` yapıyor.
3. **Direksiyonu kaç derece kırayım?** — Bu **kontrol**; sende `tracker.py`.

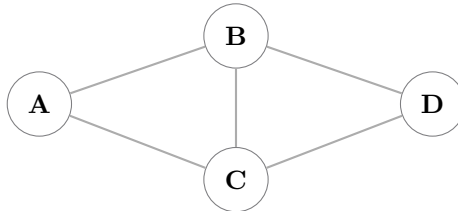
Bu belge **yalnız 1. soruyla** ilgilenir. Dijkstra, “hangi yollardan gideyim” sorusunun cevabıdır. 2. ve 3. soru ayrı derslerin konusu.

Ve 1. soruyu cevaplayabilmek için önce haritayı bilgisayarın anlayacağı bir şeye çevirmemiz lazım. O şeyin adı **graf**.

## 2 Graf nedir? (Gerçekten sıfırdan)

Bir kâğıda birkaç nokta koy. Bazılarını çizgiyle birleştir. Bitti — graf budur. İki parçası var:

- **Düğüm** (İng. *node / vertex*): noktalar. Bizde bir düğüm, pistin üzerindeki bir **koordinattır**:  $(x, y)$ .
- **Kenar** (İng. *edge*): noktaları birleştiren çizgiler. Bizde bir kenar, “bu iki nokta arasında **sürülebilir yol** var” demektir.



4 düğüm, 5 kenar — “A’dan B’ye ve C’ye gidilebilir; A’dan D’ye doğrudan gidilemez”

### 2.1 Ağırlık: kenarın bedeli

Şimdi kritik ekleme. Bir kenara sadece “var” demek yetmez — *ne kadara mal olduğunu* da bilmeliyiz. Kenarın üstüne bir sayı yazarız; buna **ağırlık** (İng. *weight*) veya **maliyet** (*cost*) denir.

## SEZGİ

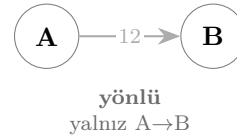
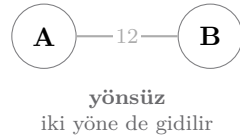
Ağırlık “uzunluk” olmak zorunda değil. Ne *istemediğini* sayıya çevirirsin:

- Ağırlık = metre → en *kısa* yol bulunur
- Ağırlık = saniye → en *hızlı* yol bulunur (trafik dahil)
- Ağırlık = metre, ama bozuk yolda  $\times 3$  → en *konforlu* yol bulunur

BURST’te ağırlık **metredir**, ama tercihli bölgelerde bir çarpanla ölçeklenir — 5. Bölümde göreceksin.

## 2.2 Yönlü graf: tek yönlü sokaklar

Kenarı çizgi yerine **ok** yaparsak, “A’dan B’ye gidilir ama B’den A’ya gidilmez” diyebiliriz. Buna **yönlü graf** denir. Tek yönlü sokak tam olarak budur.



## BURST’te durum

Bizim haritamız **yönlüdür** (`directed: true`). Sebebi `mission.py`’nin kendi yorumunda yazıyor: zincirler *trafik akış yönünde* çizildi (sağdan akış), dolayısıyla ters yön rotası kökten imkânsızdır. Araç kırmızı ışıktaki ters şeride giremez — bunu koda “o kenar yok” diye söyleriz, sonradan ceza vererek değil.

## 2.3 Grafi bilgisayara nasıl anlatırız: komşuluk listesi

Kâğıtta çizim güzel ama koda çizim koyamayız. En yaygın gösterim **komşuluk listesi** (*adjacency list*): her düğüm için “benden kimlere gidilir” listesi.

```
# Yukardaki 4 düğümlü graf, gıllarlıklarla:
adj = {
    'A': [('B', 4), ('C', 2)],          # A'dan B'ye 4, C'ye 2
    'B': [('A', 4), ('C', 1), ('D', 5)],
    'C': [('A', 2), ('B', 1), ('D', 8)],
    'D': [('B', 5), ('C', 8)],
}
# Yönlü olsaydı ters yönleri İEKLEMEZDK. Yönsüz grafta her kenar
# iki kere görünür - bu bir hata değil, 11tanmm göreli.
```

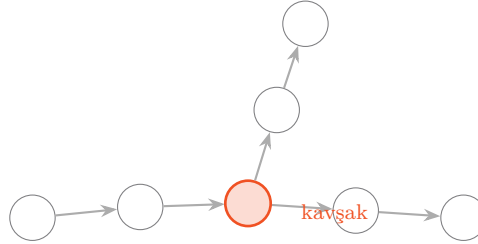
BURST’te tam olarak bu yapılıyor, sadece isimler yerine tam sayı kimlikleri (vertex id) var ve komşuluk bir küme:

```
# mission.py, __init__ içinden (şşsadeletirilmli)
self.adj = {}          # a -> {b, c, ...}   "a'dan kimlere gidilir"
self.pred = {}         # b -> {a, ...}     "b'ye kimlerden gelinir"
for a, b in vgraph['edges']:
    self.adj.setdefault(a, set()).add(b)
    self.pred.setdefault(b, set()).add(a)
    if not self.directed:          # eski yönsüz haritalar için
        self.adj.setdefault(b, set()).add(a)
        self.pred.setdefault(a, set()).add(b)
```

**Neden pred de tutuluyor?** Çünkü “bu düğüme kaç yoldan gelinir” bilgisi, kavşak olup olmadığını anlamak için gerekiyor. Girişi 2+ olan ya da çıkışı 2+ olan düğüm bir *karar noktasıdır*.

## 2.4 Bizim gerçek grafımız

Merak ediyorsan: `maps/burst_map.json` içindeki gerçek harita şu an **863 düğüm** ve **919 kenar** içeriyor, yönlü. Yani yukarıdaki 4 düğümlü oyuncak grafın aynısı, sadece 200 katı büyük.



*Gerçek harita böyle parçalardan oluşur: düğümler  $\sim 1-3$  m aralıklı, kavşakta dallanır. 863 düğüm = pistin tamamı.*

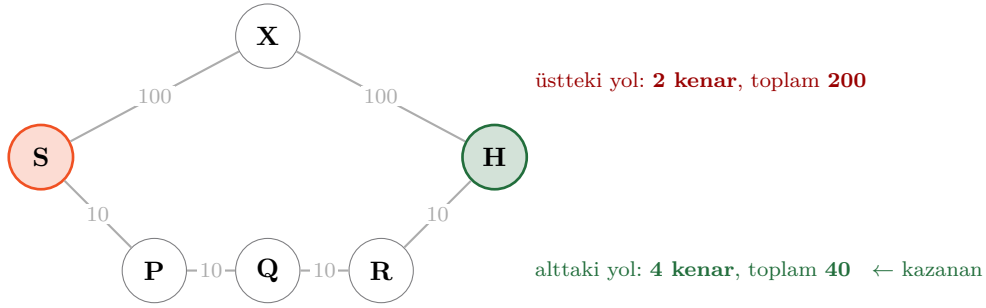


# 1. BÖLÜM — Algoritma

## 3 Problem tam olarak nedir?

**En kısa yol problemi:** Ağırlıklı bir graf, bir *başlangıç* düğümü ve bir *hedef* düğümü verildi. Başlangıçtan hedefe giden, üzerindeki ağırlıkların **toplamı en küçük** olan yolu bul.

Dikkat: “en az kenar” ile “en kısa” aynı şey *değildir*. Bu ayrımı kaçırmak en sık yapılan hatadır:



Az duraklı yol, uzun yol olabilir. Bizi ilgilendiren **toplam ağırlıktır**.

## 4 Neden “hepsini dene” işe yaramaz

En doğal fikir: bütün yolları listele, toplamalarını hesapla, en küçüğünü seç. Küçük grafta çalışır. Sonra çöker.

### Neden çöker — sayıyla

Şöyle bir harita düşün: 10 kavşak sırayla dizilmiş ve her kavşakta 2 seçenek var (sağ/sol). Yol sayısı:

$$2 \times 2 \times \cdots \times 2 = 2^{10} = 1024$$

Fena değil. 20 kavşak olsun:  $2^{20} \approx 1$  milyon. 30 kavşak:  $2^{30} \approx 1$  milyar.

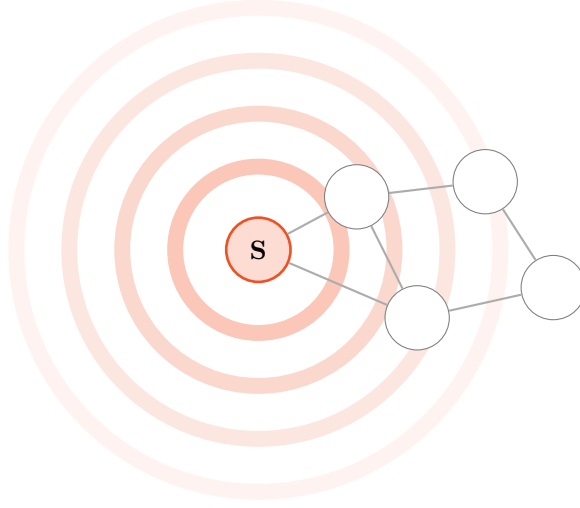
Bizim haritamızda 863 düğüm var ve döngüler de mümkün. Yol sayısı sonlu bile değil (döngüde sonsuz tur atabilirsin). “Hepsini dene” burada bir strateji değil, bir şaka.

### SEZGİ

Doğru fikir şu: yolları tek tek denemek yerine, **her düğüme kadar olan en iyi mesafeyi** biriktir ve bir daha o düğüme bakma. Böylece iş, “yol sayısı” ile değil “**düğüm sayısı**” ile büyür. 1,000,000,000 yerine 863.

## 5 Sezgi: su baskını

Dijkstra’yı ilk kez öğrenirken formülleri unut, şu resmi kafana kaz:



Başlangıç düğümüne su dök.  
Su bütün borulardan  
aynı hızda yayılsın.

Suyun bir düğüme ilk  
ulaştığı an, o düğüme olan  
en kısa mesafedir.

Su, uzun boruyu dolaşıp geç gelmez — her zaman en kısa güzergâhtan gelir. Dijkstra bu su baskını kâğıt üzerinde *simüle eder*. Tek fark: su sürekli akar, biz ise “bir sonraki suyun ulaşacağı düğüm hangisi?” sorusunu sorup adım adım zıplıyoruz.

### Dijkstra’nın tek büyük fikri:

Henüz kesinleşmemiş düğümler arasından **mesafesi en küçük olanı** seç. O düğümün mesafesi artık **kesindir** — bir daha asla değişmeyecek.

Neden kesin? Çünkü ona daha kısa bir yoldan ulaşmak isteseydin, o yol başka bir düğümden geçmek zorundaydı — ve o düğümün mesafesi zaten *daha büyük* (yoksa onu seçerdin). Ağırlıklar negatif olmadığı için daha büyük bir mesafeye bir şey ekleyerek küçüğe inemezsin. Bu argümanı 2. Bölümde tam olarak yazacağız.

## 6 Algoritma — iki tanımla üç adım

Önce iki kavramı isimlendirelim:

- **dist[v]** — “v’ye şu ana kadar bulabildiğim en iyi mesafe.” Başlangıçta  $\text{dist}[\text{başlangıç}] = 0$ , diğer herkes  $\infty$ . Bu bir *tahmindir* ve düşe düşe gerçek değerine iner.
- **Kesin küme** — mesafesi artık nihai olan düğümler. Bir düğüm buraya girdi mi bir daha ona dokunulmaz.

### DIJKSTRA — Adım Adım

#### Hazırlık

1.  $\text{dist}[s] \leftarrow 0$ , diğer tüm düğümler için  $\text{dist}[v] \leftarrow \infty$ .
2. Kesin küme boş.

**Döngü** — kesinleşmemiş düğüm kaldığı sürece tekrarla:

- A. Seç:** Kesinleşmemişler arasından **dist** değeri *en küçük* olan düğümü al; adı *u* olsun. Hepsini  $\infty$  ise dur (kalanlara ulaşamıyor).
- B. Kesinleştir:** *u*’yu kesin kümeye koy. *u* hedefse **dur** — cevabı buldun.
- C. Gevşet:** *u*’nun her komşusu *v* için *yeni bir yol denemesi* yap:

$$\text{aday} = \text{dist}[u] + w(u, v)$$

Eğer  $\text{aday} < \text{dist}[v]$  ise, daha iyi bir yol bulmuşsun demektir:

$$\text{dist}[v] \leftarrow \text{aday}, \quad \text{prev}[v] \leftarrow u$$

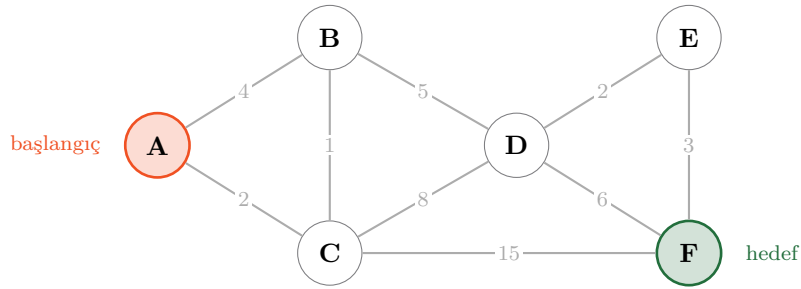


“Gevşetme” (relaxation) ne demek? Lastik bant gibi düşün:  $\text{dist}[v]$  gergin duran bir tahmindir, her yeni keşifte biraz daha *gevşer* (küçülür). Asla büyümmez. İsim buradan geliyor.

**prev ne işe yarar?**  $\text{dist}$  sana “kaç metre” söyler ama “hangi yoldan” söylemez.  $\text{prev}[v]$ , “ $v$ ’ye gelen en iyi yolda benden önceki düğüm kimdi” bilgisini saklar. Hedeften başlayıp  $\text{prev}$  zincirini geri takip edince yol ortaya çıkar. Bunu §13 ’de yapacağız.

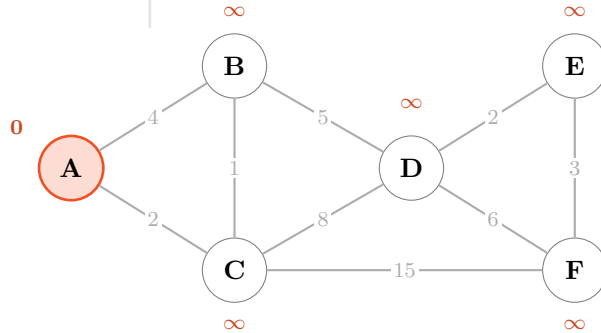
## 7 ÖRNEK 1 — Elle tam çözüm (bunu mutlaka kâğıtta tekrarla)

İşte üzerinde çalışacağımız graf. **A**’dan başlıyoruz, hedefimiz **F**. Kenarlar yönsüz (iki yöne de gidilir).



**Gözünle bir tahmin yap:**  $A \rightarrow B \rightarrow D \rightarrow F$  kaç eder?  $4 + 5 + 6 = 15$ . Peki daha iyisi var mı? Aşağıda göreceğiz — cevap **13** ve yol muhtemelen tahmin ettiğin gibi değil.

### Adım 0 — Hazırlık

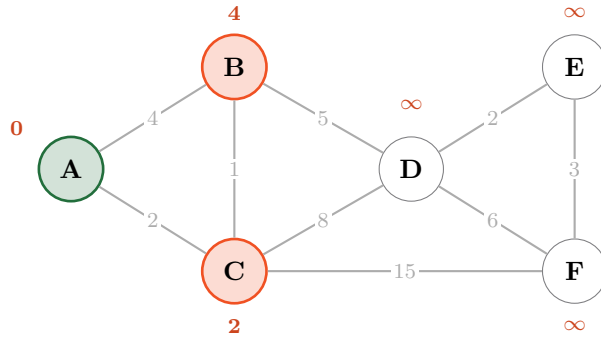


Kendine olan mesafe sıfır; kimseye nasıl gidileceğini henüz bilmiyoruz.

### Adım 1 — A kesinleşir

Kesinleşmemişlerin en küçüğü: A (0). Kesin kümeye alıyoruz ve komşularını gevşetiyoruz.

- B:  $0 + 4 = 4 < \infty \Rightarrow \text{dist}[B]=4, \text{prev}[B]=A$
- C:  $0 + 2 = 2 < \infty \Rightarrow \text{dist}[C]=2, \text{prev}[C]=A$



yeşil = kesin    turuncu = sınırdaki (keşfedildi, kesinleşmedi)

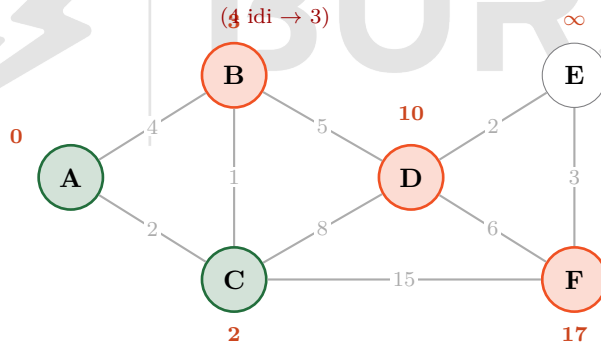
## Adım 2 — C kesinleşir (buradaki hamle önemli)

Sınırdakiler: B(4), C(2). En küçüğü C. Kesinleştir, komşularını gevşet:

- B:  $2 + 1 = 3$ . Ama  $\text{dist}[B]$  zaten 4 idi!  $3 < 4$  olduğu için **güncelliyoruz**:  $\text{dist}[B]=3$ ,  $\text{prev}[B]=C$ .
- D:  $2 + 8 = 10 < \infty \Rightarrow \text{dist}[D]=10$ ,  $\text{prev}[D]=C$
- F:  $2 + 15 = 17 < \infty \Rightarrow \text{dist}[F]=17$ ,  $\text{prev}[F]=C$

### SEZGİ

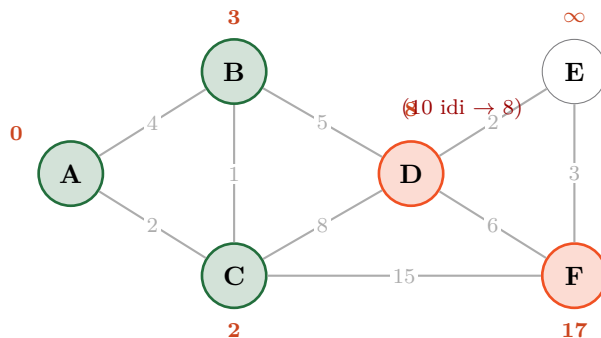
İşte gevşetmenin bütün hikâyesi bu satırda. B'ye “doğrudan” 4'e gidiliyordu; C üzerinden dolaşınca 3'e düştü. Dolambaçlı görünen yol daha kısa çıkabilir — algoritma bunu senin adına fark eder.



## Adım 3 — B kesinleşir

Sınırdakiler: D(10), F(17). En küçüğü B(3).

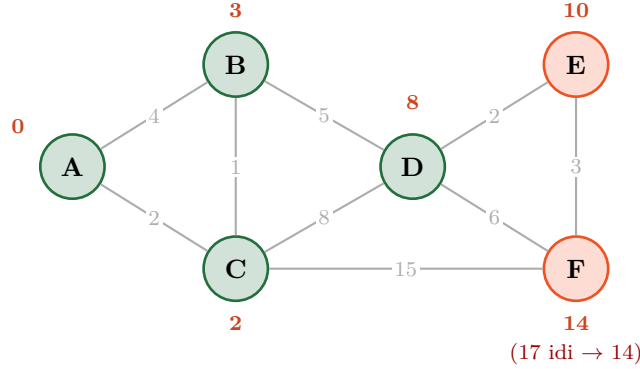
- A ve C zaten kesin — *atla* (kesin düğüme bir daha dokunulmaz).
- D:  $3 + 5 = 8$ .  $\text{dist}[D]$  10 idi,  $8 < 10 \Rightarrow \text{dist}[D]=8$ ,  $\text{prev}[D]=B$ .



### Adım 4 — D kesinleşir

Sınırdakiler: D(8), F(17). En küçüğü **D(8)**.

- E:  $8 + 2 = 10 < \infty \Rightarrow \text{dist}[E]=10, \text{prev}[E]=D$
- F:  $8 + 6 = 14$ .  $\text{dist}[F]$  17 idi,  $14 < 17 \Rightarrow \text{dist}[F]=14, \text{prev}[F]=D$ .



#### TUZAK

Burada durup “F’yi buldum, 14” **diyemezsin**. F henüz *kesin değil*, sadece sınırdadır. Sınırdaki 10 değerli E duruyor ve E üzerinden F’ye daha kısa gidilebilir — nitekim gidiliyor. **Hedefi bulduğunda değil, hedefi kesinleştirdiğinde durursun**. Bu ayırım, gerçek kodda en sık yapılan hatadır.

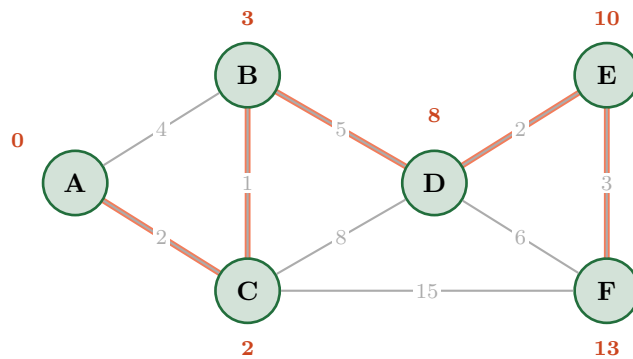
### Adım 5 — E kesinleşir

Sınırdakiler: E(10), F(14). En küçüğü **E(10)**.

- F:  $10 + 3 = 13$ .  $\text{dist}[F]$  14 idi,  $13 < 14 \Rightarrow \text{dist}[F]=13, \text{prev}[F]=E$ .

### Adım 6 — F kesinleşir: DUR

Sınırdaki yalnız F(13) kaldı; en küçük o. Kesinleştiriyoruz — ve F hedef olduğu için **duruyoruz**. Cevap: **13**.



$$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F = 2 + 1 + 5 + 2 + 3 = 13$$

Başta gözünle tahmin ettiğin  $A \rightarrow B \rightarrow D \rightarrow F$  15 ediyordu. Algoritma 13’ü buldu — hem de *daha fazla* kenar kullanan bir yolla. İnsan sezgisi en kısa yolda güvenilirdir; algoritma güvenilirdir.

## Her şeyin özeti: tek tablo

| Adım | Kesinleşen | A | B        | C        | D        | E        | F        |
|------|------------|---|----------|----------|----------|----------|----------|
| 0    | —          | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 1    | A (0)      | 0 | 4        | 2        | $\infty$ | $\infty$ | $\infty$ |
| 2    | C (2)      | 0 | 3        | 2        | 10       | $\infty$ | 17       |
| 3    | B (3)      | 0 | 3        | 2        | 8        | $\infty$ | 17       |
| 4    | D (8)      | 0 | 3        | 2        | 8        | 10       | 14       |
| 5    | E (10)     | 0 | 3        | 2        | 8        | 10       | 13       |
| 6    | F (13) DUR | 0 | 3        | 2        | 8        | 10       | 13       |

**Kalın** = kesinleşmiş, **kırmızı** = o adımda gevşeyen değer. Tabloyu soldan sağa değil, *yukarıdan aşağı* oku: her sütun bir düğümün tahmininin nasıl düştüğünü gösterir. Hiçbir sayının *arttığını* göremezsin — gevşetme tek yönlüdür.

## SAVUNMA SORUSU

“Adım 4’te F=14 bulmuştun, neden orada durmadın?” Cevap: F sınırdaydı, kesin değildi. Dijkstra bir düğümü ancak *kuyruğun en küçüğü olarak çıkardığında* kesinleştirir. Adım 4’te kuyruğun en küçüğü E(10) idi ve E, F’yi 13’e indirdi.



## 2. BÖLÜM — Neden Doğru Çalışıyor?

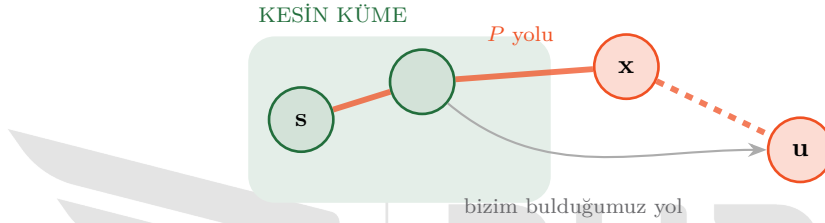
Algoritmayı uygulayabilmek bir şey, *neden* işe yaradığını bilmek başka şey. Bu bölüm kısa ama savunmada sorulacak yer burası.

### 8 Ana iddia ve kanıtı

**İddia:** Bir düğüm kesin kümeye girdiği anda, `dist` değeri gerçek en kısa mesafedir. Sonradan asla düşmez.

**Kanıt (çelişkiyle, lise seviyesi).** Diyelim ki iddia yanlış ve kesinleştirdiğimiz  $u$  düğümü için gerçek en kısa mesafe `dist[u]`'dan *küçük*. O hâlde başlangıçtan  $u$ 'ya giden, bizim bulduğumuzdan daha kısa gerçek bir  $P$  yolu vardır.

$P$  yolu, kesin kümeden çıkıp kesinleşmemiş bölgeye bir yerde geçmek zorundadır (çünkü başlangıç kesin,  $u$  ise henüz yeni kesinleşiyor). Bu geçişteki ilk kesinleşmemiş düğüme  $x$  diyelim:



Şimdi iki gözlem:

1.  $x$  de kesinleşmemiş,  $u$  da. Biz  $u$ 'yu seçtik — demek ki  $\text{dist}[u] \leq \text{dist}[x]$ . (Adım A: en küçüğü seçiyoruz.)
2.  $P$  yolu  $x$ 'ten geçip  $u$ 'ya devam ediyor. Devam eden parçanın uzunluğu negatif olamaz, çünkü **bütün ağırlıklar  $\geq 0$** . Yani

$$\text{uzunluk}(P) \geq \text{dist}[x] \geq \text{dist}[u].$$

Ama biz  $P$ 'nin `dist[u]`'dan *kısa* olduğunu varsaymıştık. Çelişki. Demek ki böyle bir  $P$  yoktur. ■

#### SEZGİ

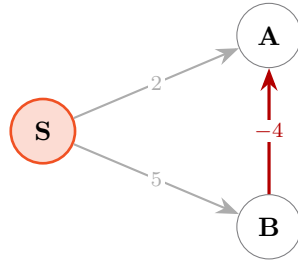
Kanıtın tamamı tek cümleye iniyor: “**Daha uzak bir yerden dolaşarak daha yakına varamazsın — çünkü yollar seni geri götürmüyor.**” Bu cümlenin doğru olması için ağırlıkların negatif olmaması şart. Bir sonraki bölüm tam olarak bunu gösteriyor.

### 9 Negatif ağırlık: Dijkstra'nın tek ölümcül zaafı

#### TUZAK

Bir kenarın ağırlığı **negatif**se Dijkstra sessizce yanlış cevap verir. Çökmez, hata vermez — sadece yanlış yolu döndürür. Bu yüzden tehlikelidir.

En küçük karşı örnek üç düğümden ibaret:



Dijkstra: S'ten sonra en küçük A(2), A'yı 2 diye kesinleştirir. **YANLIŞ.**

Gerçek:  $S \rightarrow B \rightarrow A = 5 + (-4) = 1$ .

Neden bozuldu? Kanıttaki 2. gözlem çöktü:  $x = B$  düğümünün mesafesi (5)  $u = A$ 'nınkinden (2) büyüktü, ama devamındaki kenar *negatif* olduğu için toplam yine de küçüldü. “Uzaktan dolaşıp daha yakına varmak” negatif kenarla mümkün hâle geliyor.

**Sıfır ağırlık sorun mu?** Hayır. Kanıtta “ $\geq 0$ ” dedik, “ $> 0$ ” demedik. Sıfır uzunluklu kenar Dijkstra’yı bozmaz — BURST’ün grafında 0.5 m altı yüzlerce mikro-kenar var ve sorun çıkarmıyorlar.

### BURST’te bu kural nerede karşımıza çıkıyor?

`mission.py` içinde `prefer_zones` diye bir özellik var: belirli bir dikdörtgenin içinden geçen kenarların maliyeti bir *çarpanla* ölçekleniyor (yarışta tüneli kazandırmak için). Kodun yorumunda aynen şu yazıyor:

*“factor > 0 şart (Dijkstra negatif maliyet kabul etmez)”*

Dikkat et: tüneli tercih ettirmek için maliyeti **çıkarmıyoruz**,  $0 < f < 1$  olan bir çarpanla **ölçekliyoruz**. “–200 metre bonus” verseydik graf negatif kenar kazanır, rota planlayıcı sessizce yanlış rotalar üretmeye başlardı. Ödül vermek istediğinde **çıkarma yapma, çarp**.

### SAVUNMA SORUSU

“Ağırlıkları negatif yapmak yerine hepsine sabit bir sayı eklesem (mesela +10) negatiflik biter, sorun çözülür mü?” **Hayır.** Çünkü sabit eklemek, *çok kenarlı* yolları orantısız cezalandırır: 4 kenarlı yol +40, 2 kenarlı yol +20 alır. En kısa yolun kendisi değişir. Sabit kaydırma en kısa yolu korumaz.

## 3. BÖLÜM — Kod

### 10 Sürüm 1: en yalın hâli (yığın yok)

Önce doğrudan sözde koddan çevirelim. Yavaş ama okunaklı — ve *doğru*. Bir algoritmayı önce böyle yazarsın, sonra hızlandırırsın.

```
def dijkstra_naif(adj, baslangic, hedef):
    """adj: {dugum: [(komsu, agirlik), ...]}   agirlik >= 0 olmalı."""
    dist = {v: float('inf') for v in adj}    # herkes sonsuz uzakta
    prev = {}                                # yolu geri kurmak için
    dist[baslangic] = 0.0
    kesin = set()

    while True:
        # --- ADIM A: kesinlesmemislerin en kucugunu bul (dogrusal tarama)
        u, en_iyi = None, float('inf')
        for v in adj:
            if v not in kesin and dist[v] < en_iyi:
                u, en_iyi = v, dist[v]

        if u is None or en_iyi == float('inf'):
            break                               # kalan herkes ulasilamaz -> bitti
        if u == hedef:
            break                               # ADIM B: hedef kesinlesti -> cevap hazir

        kesin.add(u)

        # --- ADIM C: komsulari geuset
        for v, w in adj[u]:
            if v in kesin:
                continue                         # kesinlesmise dokunma
            aday = dist[u] + w
            if aday < dist[v]:
                dist[v] = aday                  # daha iyi yol bulundu
                prev[v] = u

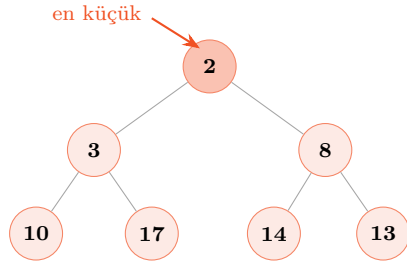
    return dist, prev
```

Bu 25 satır, 1. Bölümdeki *metotbox*’ın birebir çevirisidir. Yan yana koyup satır satır eşleştir — “Adım A”, “Adım B”, “Adım C” yorumları tam da o adımların yerini gösteriyor.

### 11 Neden yavaş? Ve öncelik kuyruğu

Sorun Adım A’da: *her turda* bütün düğümleri tarayıp en küçüğü arıyoruz.  $V$  düğüm varsa  $V$  tur  $\times$   $V$  tarama =  $V^2$  iş.

Bize şunu yapan bir veri yapısı lazım: “bir sürü sayı tut, istediğimde **en küçüğünü** çabuk ver.” Bunun adı **öncelik kuyruğu** (*priority queue*); Python’daki gerçekleştirilmesi *heapq* (ikili yığın).



**Yığın kuralı:** her düğüm çocuklarından küçüktür.

Bu yüzden **en küçük her zaman tepededir** — bakması bedava.

Ekleme ve çıkarma:  $\log_2 n$  adım.

863 elemanlı bir yığında en küçüğü bulmak 863 karşılaştırma değil, yaklaşık  $\log_2 863 \approx 10$  karşılaştırma sürer.

## 12 Sürüm 2: yığın — gerçek kullanacağın sürüm

```
import heapq

def dijkstra(adj, baslangic, hedef=None):
    """En kısa yol. Donen: (dist, prev). Ağırlıklar >= 0 olmalı."""
    dist = {baslangic: 0.0}          # SADECE keşfedilenleri tutarız
    prev = {}
    pq = [(0.0, baslangic)]          # (maliyet, dugum) - heapq maliyete göre sıralar

    while pq:
        c, u = heapq.heappop(pq)      # ADIM A: en kucugu al - O(log n)

        if c > dist.get(u, float('inf')):
            continue                  # BAYAT KAYIT - asagida acikliyorum

        if u == hedef:
            break                     # ADIM B: hedef kesinlesti

        for v, w in adj[u]:           # ADIM C: geuset
            yeni = c + w
            if yeni < dist.get(v, float('inf')):
                dist[v] = yeni
                prev[v] = u
                heapq.heappush(pq, (yeni, v))    # O(log n)

    return dist, prev
```

### 12.1 “Bayat kayıt” satırı — burayı anlamadan geçme

#### TUZAK

heapq’nun “içindeki bir değeri güncelleme” diye bir işlemi yoktur. B’nin maliyetini 4’ten 3’e indirdiğinde, yığındaki eski  $(4, B)$  kaydını silemezsin.

Çözüm basit ve zariftir: **silme, üstüne yeni kayıt at**. Yığında bir süre hem  $(4, B)$  hem  $(3, B)$  durur. Küçük olan  $(3, B)$  önce çıkar ve işlenir. Sonra sıra  $(4, B)$ ’ye geldiğinde:

$c = 4$      $\text{dist}[B] = 3$



$4 > 3 \Rightarrow$  bu kayıt bayat, `continue` ile atla.



Bu tekniğe **tembel silme** (*lazy deletion*) denir. Yığın biraz şişer ama kod çok basitleşir ve karmaşıklık değişmez. **BURST’ün kodunda da aynen bu satır var** — 5. Bölümde göstereceğim.

## 13 Yolu geri kurmak

`dist` sana “13 metre” der; “hangi yoldan” demez. Onu `prev` söyler. Hedeften başlayıp geriye zincir takip edersin:



prev zincirini takip et ... sonra listeyi ters çevir

```

def yolu_kur(prev, baslangic, hedef):
    """prev zincirini geri sararak dugum listesi uredir."""
    if hedef != baslangic and hedef not in prev:
        return [] # ulasilamaz
    yol = [hedef]
    while yol[-1] != baslangic:
        yol.append(prev[yol[-1]]) # bir adim geriye
    yol.reverse() # F,E,D,B,C,A -> A,C,B,D,E,F
    return yol
  
```

Örnek 1’i bu kodla çalıştırsan çıktı: `['A', 'C', 'B', 'D', 'E', 'F']`, `dist['F'] = 13`. Kâğıtta bulduğumuzun aynısı.

## 14 Ne kadar hızlı? BURST’ün sayılarıyla

| Sürüm         | Karmaşıklık       | Bizim harita ( $V=863$ , $E=919$ )     |
|---------------|-------------------|----------------------------------------|
| Naif (tarama) | $O(V^2)$          | $863^2 \approx 745\,000$ işlem         |
| Yığınlı       | $O((V+E) \log V)$ | $1782 \times 10 \approx 17\,800$ işlem |

Yaklaşık **40 kat** fark. “Küçük harita, fark etmez” deme: rota planlayıcı sabit değil, engel çıktığında veya tabela okunduğunda *yeniden* plan yapıyor. Saniyede birkaç kez çalışan bir hesapta 40 kat, CPU’nun algıya mı yoksa beklemeye mi harcandığı demektir.

**log nereden geldi?** Yığında  $n$  eleman varsa yükseklik  $\log_2 n$ ’dir — her seviyede eleman sayısı ikiye katlanır. Ekleme/çıkarma en fazla bir kez yukarıdan aşağı iner, yani  $\log_2 n$  adım. 863 için  $\approx 10$ ; 1 milyon için  $\approx 20$ . Logaritma, “kaç kere ikiye bölebilirim” sayısıdır.

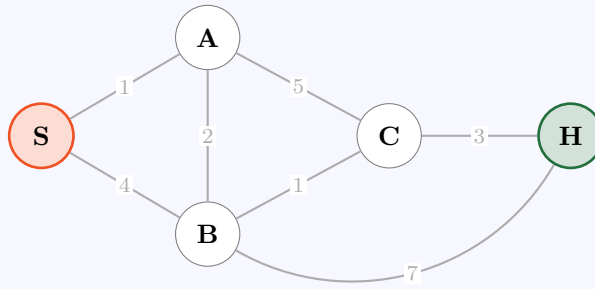
## 4. BÖLÜM — Örnek Sorular ve Tam Çözümler

Her soruyu **önce kendin çöz**, sonra çözüme bak. Kâğıt-kalem şart: tabloyu elle doldurmadan Dijkstra oturmaz. İlk 5 soru algoritmayı, son 5 soru doğrudan BURST'ü çalıştırıyor.

### 15 Soru 1 — Isınma: elle tam tablo

#### Soru

Aşağıdaki grafta S'ten H'ye en kısa yolu ve uzunluğunu bul. Tabloyu adım adım doldur.



#### Çözüm.

| Adım | Kesinleşen | S | A        | B        | C        | H        |
|------|------------|---|----------|----------|----------|----------|
| 0    | —          | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 1    | S (0)      | 0 | 1        | 4        | $\infty$ | $\infty$ |
| 2    | A (1)      | 0 | 1        | 3        | 6        | $\infty$ |
| 3    | B (3)      | 0 | 1        | 3        | 4        | 10       |
| 4    | C (4)      | 0 | 1        | 3        | 4        | 7        |
| 5    | H (7) DUR  | 0 | 1        | 3        | 4        | 7        |

Adım 2'de B, 4  $\rightarrow$  3'e düştü (A üzerinden). Adım 3'te C, 6  $\rightarrow$  4'e düştü. Adım 4'te H, 10  $\rightarrow$  7'ye düştü.

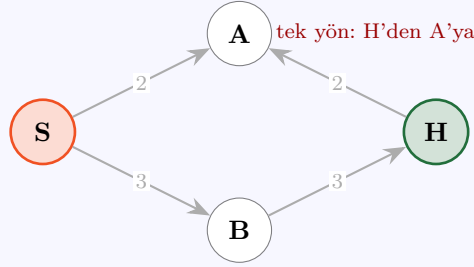
prev zinciri: H $\leftarrow$ C $\leftarrow$ B $\leftarrow$ A $\leftarrow$ S. Yol: S  $\rightarrow$  A  $\rightarrow$  B  $\rightarrow$  C  $\rightarrow$  H, uzunluk 7.

S'ten B'ye doğrudan bir kenar (4) olmasına rağmen çözüm A üzerinden dolaşp 3'e indi. “Doğrudan kenar en kısadır” diye bir kural yok.

### 16 Soru 2 — Tek yön: ters şeride giremezsin

#### Soru

Bu graf **yönlüdür** (oklara dikkat). S'ten H'ye en kısa yol nedir?



**Çözüm.** Göz “ $S \rightarrow A \rightarrow H = 4$ ” demek istiyor. Ama  $A \rightarrow H$  kenarı **yok**; olan  $H \rightarrow A$ . Yönlü grafta ok yönünün tersine gidilmez.

- Pop S(0):  $A=2$ ,  $B=3$
- Pop A(2): A’nın *çıkışı yok* ( $\text{adj}[A]$  boş) — gevşetecek kimse yok, çıkmaz.
- Pop B(3):  $H = 3 + 3 = 6$
- Pop H(6): hedef, dur.

**Cevap:**  $S \rightarrow B \rightarrow H$ , **uzunluk 6**. A’ya girmek 2’ye mal oluyor ama oradan çıkış olmadığı için ölü yatırım.

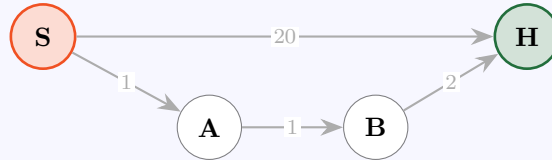
#### Bu neden birebir BURST

Haritamız yönlü ve zincirler trafik akış yönünde çizildi. Araç hedefi “arkasında” görse bile ters şeritten gidemez — pistin ileri yönünde dolanıp gelmek zorundadır. `mission.py` bunu ceza vererek değil, **o kenarı hiç eklemeyerek** sağlar.

## 17 Soru 3 — “Hedefi gördüm, durabilir miyim?”

### Soru

Aşağıdaki grafta ilk adımda H keşfediliyor ve  $\text{dist}[H] = 20$  oluyor. Algoritma burada durup “cevap 20” diyebilir mi? Doğru cevap kaç?



**Çözüm.** Hayır, **duramaz**. H keşfedildi ama *kesinleşmedi*; sınırdadır.

- Pop S(0):  $H=20$ ,  $A=1$
- Pop A(1) — kuyruğun en küçüğü A, H değil:  $B=2$
- Pop B(2):  $H = 2 + 2 = 4 < 20 \Rightarrow H=4$
- Pop H(4): şimdi kesinleşti, dur.

**Cevap:** 4 ( $S \rightarrow A \rightarrow B \rightarrow H$ ).

**Kural:** Hedefi *gevşetirken* değil, **kuyruktan çıkarırken** dur. Kodda fark tek satır: `heappop` sonrası `if u == hedef: break` — `heappush` öncesi değil.

## 18 Soru 4 — Beraberlik: iki yol da 5

### Soru

$S \rightarrow A \rightarrow H = 3 + 2 = 5$  ve  $S \rightarrow B \rightarrow H = 3 + 2 = 5$ . Dijkstra hangisini döndürür?

**Çözüm. İkisi de doğrudur.** En kısa yol *tek* olmak zorunda değil; tek olan şey en kısa *maliyettir* (5).

Hangisinin döneceği, yığından hangisinin önce çıktığına bağlıdır — yani düğümleri hangi sırayla eklediğine. Python’da (*maliyet*, *dugum*) demeti sıralanırken maliyetler eşitse *düğüm* karşılaştırılır; bu yüzden sonuç *deterministiktir* ama “anlamli” değildir.

### TUZAK

Demetin ikinci elemanı karşılaştırılmaz bir tipse (mesela bir sınıf örneği) Python beraberlikte `TypeError` fırlatır. Bu yüzden kuyruğa (*maliyet*, *sayac*, *nesne*) koymak yaygın bir kalıptır. BURST’te durum (*int*, *int*) demeti olduğu için sorun yok.

Yarıшта beraberliği şansa bırakmak istemezsin. Çözüm: maliyete küçük ve *anlamli* bir ayırt edici ekle — örneğin dönüş sayısı. BURST’te U-dönüş cezası (*uturn\_penalty\_m*) tam olarak böyle bir ayırt edicidir.

## 19 Soru 5 — Ulaşılamayan hedef

### Soru

Hedef düğüme giden hiçbir kenar yoksa algoritma ne yapar? Sonsuza kadar döner mi?

**Çözüm.** Hayır. Yığın *boşalır* ve `while pq` döngüsü kendiliğinden biter. `dist` sözlüğünde hedef hiç görünmez (veya  $\infty$  kalır), `prev`’te de yoktur — `yolu_kur` boş liste döndürür.

Kritik nokta **bunu çağıran tarafın anlaması**. BURST’te bu durum sessizce geçilmez, açık bir gerekçeyle raporlanır:

```
if goal_state is None:
    return RouteState(valid=False, ..., reason='rota bulunamad (blok/kural?)')
```

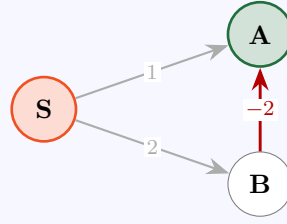
### SAVUNMA SORUSU

“Rota bulunamadı” üç sebepten olabilir: (1) hedef gerçekten kopuk, (2) bütün geçerli kenarlar bloklanmış, (3) tabela kuralları tüm çıkışları yasaklamış. Kod bu üçünü *reason* alanında ayırt edebiliyor mu? Ayıramıyorsa sahada hangi hatayı arayacağımı bilemezsin.

## 20 Soru 6 — Negatif ağırlık: yanlış yakala

### Soru

Bu grafta Dijkstra S’ten A’ya kaç bulur? Doğru cevap kaç? Nerede hata yaptı?

**Çözüm.**

- Pop S(0): A=1, B=2
- Pop A(1) — en küçük A. Hedef A olduğu için **dur, cevap 1**.

Ama gerçek en kısa:  $S \rightarrow B \rightarrow A = 2 + (-2) = 0$ . **Dijkstra 1 dedi, doğrusu 0.**

**Hata nerede?** 2. Bölümdeki kanıtın 2. gözlemi: “devam eden parça negatif olamaz.” Burada olabildi. B’nin mesafesi (2) A’nınkinden (1) büyüktü, dolayısıyla Dijkstra “B üzerinden gelen yol daha kısa olamaz” diye varsaydı — negatif kenar bu varsayımı çürüttü.

**Negatif kenar gerekiyorsa ne kullanılır?** Bellman–Ford ( $O(V \cdot E)$ ), negatif kenara dayanıklı) veya Johnson algoritması. Bizde gerek yok: mesafe negatif olamaz, süre negatif olamaz. Sadece “ödül”ü çıkarma olarak modellemekten kaçın.

**21 Soru 7 — Ağırlığı değiştir, cevap değişsin****Soru**

İki rota var:

- **Ana yol:** 3000 m, ortalama hız 20 m/s
- **Ara sokak:** 1200 m, ortalama hız 6 m/s

(a) Ağırlık *metre* ise hangisi kazanır? (b) Ağırlık *saniye* ise?

**Çözüm.**

(a) Metre:  $1200 < 3000 \Rightarrow$  **ara sokak** kazanır.

(b) Saniye:  $t = d/v$

$$t_{\text{ana}} = \frac{3000}{20} = 150 \text{ s}, \quad t_{\text{ara}} = \frac{1200}{6} = 200 \text{ s}$$

$150 < 200 \Rightarrow$  **ana yol** kazanır.

Algoritma değişmedi — **maliyet fonksiyonu** değişti ve cevap tersine döndü. Dijkstra “en kısa yolu” bulmaz; **sen ne dersen onun en azını** bulur. Sorumluluk ağırlığı tanımlayanda.

**22 Soru 8 — Tercihli bölge: tüneli kazandırmak**

**Soru**

Yarıştta tünelden geçmek ekstra puan getiriyor, o yüzden araç makul bir sapmayı göze alsın istiyoruz.

- Tünel rotası: **250 m**
- Yüzey rotası: **180 m**

(a) Hiçbir şey yapmazsak hangisi seçilir? (b) Tünel kenarlarına  $f = 0.4$  çarpanı uygularsak? (c) Çarpan yerine “-200 m bonus” verseydik ne olurdu?

**Çözüm.**

(a)  $180 < 250 \Rightarrow$  **yüzey** seçilir. Tünel hiç kullanılmaz.

(b) Tünelin *görünen* maliyeti:

$$250 \times 0.4 = 100$$

$100 < 180 \Rightarrow$  **tünel** seçilir. Ağırlıklar hâlâ pozitif, Dijkstra sapasağlam.

(c)  $250 - 200 = 50$ . Tünel yine kazanır *ama* artık bir bomba kurdun: bonus, üzerine uygulandığı kenardan büyükse ağırlık negatife düşer. Örneğin tünel içindeki 150 m’lik bir kenara aynı bonusu uygularsan  $150 - 200 = -50$  olur ve Soru 6’daki sessiz yanlışlık başlar.

**Koddaki karşılığı**

```
def _edge_cost(self, a, b):
    """Dijkstra kenar maliyeti: uzunluk × bölge tercihi çarpan."""
    L = self._edge_len(a, b)
    if self.prefer_zones:
        mx = 0.5 * (self.verts[a][0] + self.verts[b][0]) # kenarin ORTASI
        my = 0.5 * (self.verts[a][1] + self.verts[b][1])
        for x0, y0, x1, y1, f in self.prefer_zones:
            if x0 <= mx <= x1 and y0 <= my <= y1:
                L *= f # CARPMA - cikarma degil
                break
    return L
```

Kenarın *orta noktasına* bakıyor: kenar bölgeye yarı yarıya giriyorsa ya tamamen sayılır ya hiç. Basit ve öngörülebilir; “kısmi” hesap yapmıyor.

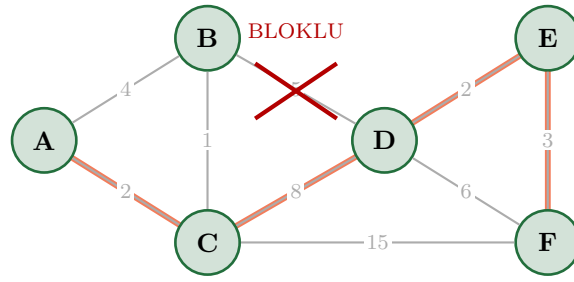
**23 Soru 9 — Engel çıktı: rotayı yeniden hesapla****Soru**

1. Bölümdeki örnek grafta (cevap 13, yol  $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$ ) **B-D kenarı bloklandı** (yolda duran araç). Yeni rota ve maliyet ne?

**Çözüm.** Bloklı kenarı “yokmuş gibi” atlarız (kodda `continue`).

- Pop A(0): B=4, C=2
- Pop C(2): B=3, D=10, F=17
- Pop B(3): B-D **bloklı**, **atla**. B’den gidilecek başka yer yok.
- Pop D(10): E=12, F=10 + 6 = 16 (< 17)
- Pop E(12): F=12 + 3 = 15 (< 16)
- Pop F(15): dur.

**Yeni yol:**  $A \rightarrow C \rightarrow D \rightarrow E \rightarrow F$ , **maliyet 15** ( $2 + 8 + 2 + 3$ ). Blok yüzünden 13’ten 15’e çıktık — 2 m’lik bedel.



yeni maliyet:  $2 + 8 + 2 + 3 = 15$  (eskisi 13 idi)

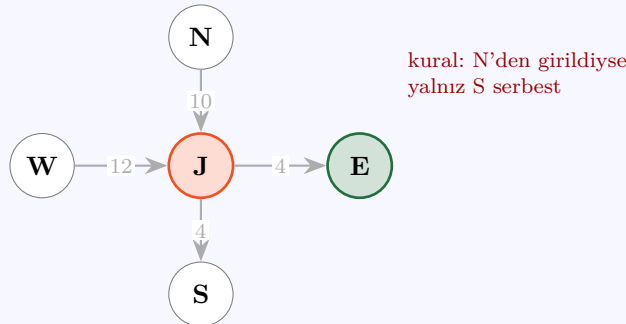
### TTL — bloklar kalıcı değildir

BURST'te blok süreli konur: `block_edge(a, b, now, ttl_sec=120)`. Sebebi basit — engel bir *park hâlindeki araç* olabilir de, *az sonra çekip gidecek* bir araç da. Süre dolunca blok düşer ve kısa rota geri kazanılır. Süresiz blok koyarsan, engel kalktıktan sonra bile araç uzun yoldan dolaşmaya devam eder.

## 24 Soru 10 — Kavşak kuralı: düğüm araması neden yetmez?

### Soru

Bir kavşak J'ye *kuzeyden* (N) ve *batıdan* (W) girilebiliyor; çıkışlar *doğu* (E) ve *güney* (S). Tabela diyor ki: “**Kuzeyden gelen ileri (S) devam etmek zorunda.**” Batıdan gelen serbest. Hedef: E'ye ulaşmak. Klasik (düğüm-bazlı) Dijkstra bu kuralı uygulayabilir mi?



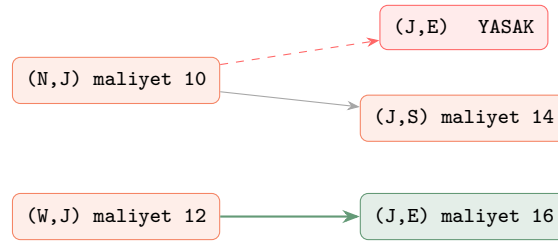
**Çözüm. Hayır, uygulayamaz — ve sebebi öğrenmen gereken asıl şey.**

Düğüm-bazlı Dijkstra her düğüm için **tek bir sayı** tutar: `dist[J]`. J'ye 10 ile (N'den) ve 12 ile (W'den) gelinebiliyor; algoritma küçüğünü saklar: `dist[J] = 10`. Ama bu sayıyı saklarken **hangi yönden geldiği bilgisini çöpe atar**. Sonra J'den çıkışları gevşetirken kuralı uygulayamaz — “kural hangi girişe göre?” sorusunun cevabı elinde yoktur.

İki kötü seçenektan birine mahkûm olur:

- Kuralı *uygular* (en iyi giriş N idi, o hâlde E yasak)  $\Rightarrow$  E'ye “rota yok” der. **Yanlış** — W üzerinden gidilebilirdi.
- Kuralı *uygulamaz*  $\Rightarrow$  14 maliyetle  $N \rightarrow J \rightarrow E$  rotasını üretir. **Yasak** — araç tabelayı ihlal eder.

**Doğru çözüm: durumu düğüm değil, yönlü kenar yap.** Yani “neredeyim” değil, “nereye hangi kenardan geldim” sakla:



İki ayrı durum, iki ayrı maliyet, iki ayrı kural kümesi.  
E'ye giden geçerli rota:  $W \rightarrow J \rightarrow E$ , maliyet 16.

**Genel ders:** Bir kısıt “nereden geldiğine” bağlıysa, arama durumuna o bilgiyi **koymak zorundasın**. Durum uzayını genişletmek (düğüm  $\rightarrow$  yönlü kenar) algoritmayı değiştirmez — Dijkstra aynı Dijkstra’dır, sadece “düğüm” dediği şey artık bir kenardır.

Bu, BURST’ün rota planlayıcısının en önemli tasarım kararıdır ve bir sonraki bölümün tamamı bunun üzerine kurulu.

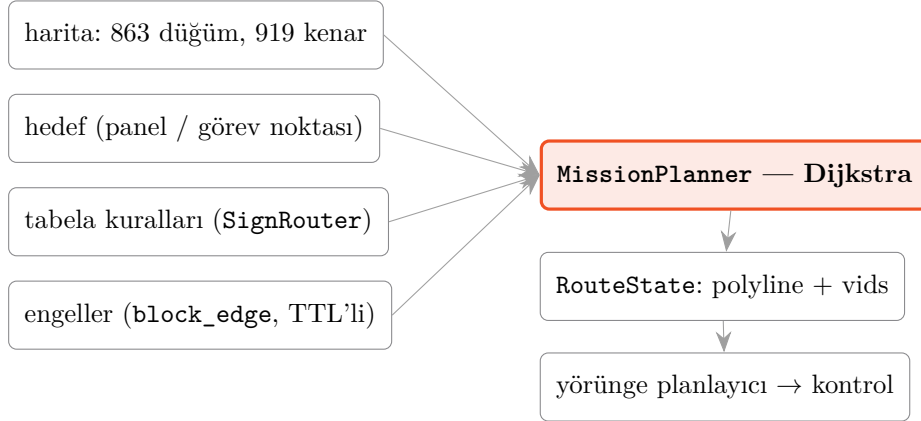




## 5. BÖLÜM — BURST: mission.py'nin İçi

Buraya kadar öğrendiğin her şey şimdi gerçek koda oturuyor. Bu bölümü bitirdiğinde burst\_planning paketindeki mission.py dosyasını açıp baştan sona okuyabiliyor olacaksın.

### 25 Rota planlayıcı yığının neresinde?



**Girdi** bir graf, bir hedef ve kısıtlar. **Çıktı** bir RouteState: takip edilecek nokta dizisi (polyline) ve üzerindeki düğüm kimlikleri. Aradaki hesap, öğrendiğin Dijkstra.

### 26 En kritik karar: durum bir *kenardır*

Soru 10'da gördüğün tasarım kararı, dosyanın en başında yazıyor:

```
"""Mission katmanı - vertex grafında donus-kisitli en kısa rota.
```

```
Arama KENAR-BAZLIDIR: Dijkstra durumu yonlu kenar (u->v). Boylece "kavsaga su kenardan girildiginde su cikislar serbest" kurallari (mecburi yon, donulmez, girisi olmayan yol) dogal ifade edilir - dugum-bazli Dijkstra bunu temsil edemez (ayni dugume farkli yonlerden farkli izinlerle gelinir)."""
```

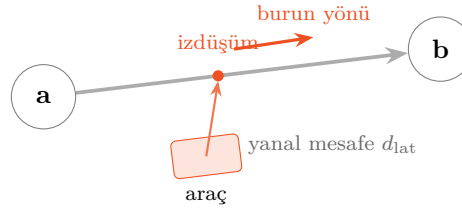
Kodda bunun görünümü şu: sözlüklerin anahtarı düğüm değil, **demet**.

```
dist = {}      # (u,v) -> maliyet   "u'dan gelerek v'ye varmanın bedeli"
prev = {}      # (u,v) -> önceki yonlu kenar
```

Durum sayısı artık düğüm sayısı değil **kenar sayısıdır**: 863 yerine 919. Neredeyse aynı — yani bu genişletme sana bedavaya geldi. Genel olarak durum uzayını genişletmek pahalıdır; burada ucuz olmasının sebebi grafın seyrek olması (her düğümün ortalama ~1.06 çıkışı var).

### 27 Başlangıç sorunu: araç düğümün üstünde değil

Ders kitabındaki Dijkstra “S düğümünden başla” der. Gerçekte araç bir düğümün üstünde durmaz — iki düğüm *arasında* bir yerdedir. Öyleyse arama nereden başlayacak?



Çözüm: aracı en yakın kenarlara **dik izdüşür** ve o kenarların *ileri uçlarını* birer başlangıç (*tohum*) olarak kuyruğa at — her birine hak ettiği başlangıç maliyetiyle.

```
for align, far, ne, L, d_lat, a, b, proj in fwd:
    align_pen = self.urn_pen * 0.5 * (1.0 - align)      # burun ne kadar donuk?
    block_pen = 1e6 if self._is_blocked(a, b, now) else 0.0
    st = (ne, far)                                     # ne->far yonlu kenari
    c = L + align_pen + block_pen + 2.0 * d_lat
    if c < dist.get(st, float('inf')):
        dist[st] = c
        seed_proj[st] = proj
        heapq.heappush(pq, (c, st))
```

Başlangıç maliyeti dört parçadan oluşuyor — her biri bir ders:

|                   |                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------|
| L                 | İzdüşümden kenarın ileri ucuna kalan mesafe. Doğal maliyet.                                                                      |
| align_pen         | Aracın burnu o kenarla ne kadar uyumsuzsa o kadar ceza. align = 1 (tam hizalı) → ceza 0; align = 0 (dik) → yarım U-dönüş cezası. |
| block_pen         | Bloklu kenar için $10^6$ . <b>Atlamıyor, cezalandırıyor — dikkat.</b>                                                            |
| $2 \cdot d_{lat}$ | Araç hangi şeride daha yakınsa o tercih edilsin.                                                                                 |

## TUZAK

**Neden bloklu tohum atlanmıyor da  $10^6$  ceza alıyor?** Çünkü araç *bloklu kenarın üstünde duruyor* olabilir. Atlarsak hiç tohum kalmaz, arama boş kuyrukla başlar ve “rota bulunamadı” der — araç kilitlenir.  $10^6$  ceza ile: başka bir tohum varsa o kazanır; hiç yoksa bloklu tohum yine de bir rota üretir. **Ceza, imkânsızlıktan iyidir.**

Tohumlar yalnız **ileri hizalı** yönlerden seçiliyor (`fwd = [s for s in seeds if s[0] > 0.0]`). Sebebi kodun yorumunda: araç yerinde dönemez. Hedef geride kalsa bile rota, pist döngüsünden *ileri* dolandır.

## 28 Ana döngü — satır satır

İşte gerçek kod. Her satırın karşısına hangi bölümde öğrendiğini yazdım.

```
goal_state = None
while pq:
    c, (u, v) = heapq.heappop(pq)      # ADIM A: en kucugu al

    if c > dist.get((u, v), float('inf')):
        continue                      # bayat kayıt (tembel silme) - 3. Bolum

    if v == self.goal_vid:             # ADIM B: hedef KESINLESTI - Soru 3
        goal_state = (u, v)
```

```

break

rule = self.rules.get((v, u))          # "v'ye u'dan girildiyse" kuralı
for w in self.adj.get(v, ()):          # ADIM C: gevset

    if w == u:                          # U-donus yasak (arac yerinde donemez)
        continue

    if rule is not None and w not in rule['allowed']:
        continue                        # tabela yasakladi - Soru 10

    if self._is_blocked(v, w, now):      # engel - Soru 9
        continue

    if (self._edge_len(u, v) > 0.7 and self._edge_len(v, w) > 0.7
        and abs(self._turn_angle(u, v, w)) > self.max_turn):
        continue                        # firkete: suremeyecegimiz rota

    nc = c + self._edge_cost(v, w)       # agirlik = uzunluk x bolge - Soru 8
    if nc < dist.get((v, w), float('inf')):
        dist[(v, w)] = nc                # GEVSETME
        prev[(v, w)] = (u, v)
        heapq.heappush(pq, (nc, (v, w)))

```

## 28.1 Dört filtre, dört gerçek dünya problemi

### 1. U-dönüş yasağı — `if w == u: continue`

Grafta “geldiğin kenardan geri dön” her zaman mümkündür ve çoğu zaman kısa görünür. Ama araç yerinde dönemez. Kodun yorumunda bunun sahada ne yaptığı yazıyor: *“turnback rotası canlıda aracı SW çıkmazına sokup kilitledi.”* Algoritmanın matematiksel olarak geçerli çözümü, fiziksel olarak sürülemez bir çözümdü.

**Sürülemeyen rota, rota değildir.** Planlayıcının ürettiği her yol, aracın gerçekten yapabileceği bir şey olmak zorunda. Bu kısıtı *arama sırasında* uygulamak, sonradan filtrelemekten hem daha ucuz hem daha doğrudur.

### 2. Tabela kuralı — `rule['allowed']`

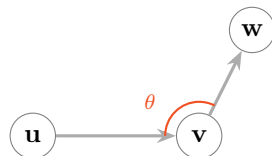
Kavşağa gelmeden önce tabelayı gördük ve `set_rule(j, in_vid, allowed)` ile “bu kavşağa bu kenardan girilince şu çıkışlar serbest” yazdık. Arama sırasında bu bir küme üyelik testine indirgeniyor. Soru 10’daki tasarımın bütün karşılığı bu tek satır.

### 3. Blok — `_is_blocked(v, w, now)`

`now` parametresine dikkat: blok *zamana bağlı*. TTL dolduysa `_is_blocked` `False` döner ve kenar kendiliğinden geri gelir.

### 4. Kinematik dönüş sınırı — `max_turn`

115°’den keskin dönüş üretilmez. Ama filtre yalnız iki kenar da 0.7 m’den uzunsa uygulanır:



$|\theta| > 115^\circ$  ise bu geçiş elenir.

Ama kenarlardan biri 0.7 m’den kısaysa filtre uygulanmaz — mikro-segmentte “yön” zaten gürültüdür.

Grafta 0.5 m altı **122 kenar** var (kavşak kümeleri). Böyle bir kenarın açısını ölçmek, 5 cm'lik bir çizginin yönünü ölçmek gibidir — gürültü. Filtreyi koşulsuz uygulaysaydın geçerli kavşak geçişlerini elerdin.

## 29 Geri sarma: kenar zincirinden sürülebilir çizgiye

prev zincirini geri sarıyoruz (§13'deki yolu\_kur ile aynı fikir), sonra kenar durumlarını noktalara çeviriyoruz:

```
chain = [goal_state]
while chain[-1] in prev:
    chain.append(prev[chain[-1]])
chain.reverse() # baslangictan hedefe dogru sirala

vids = [st[1] for st in chain] # her kenar durumunun VARIS dugumu
proj = seed_proj.get(chain[0], p) # secilen tohumun izdusum noktası
pts = [p, proj] if norm(proj - p) > 0.3 else [p] # aracın kendi konumu + izdusum
for v in vids:
    pts.append(self.verts[v])
poly = np.array(pts)
```

### SEZGİ

Polyline'ın ilk iki noktası düğüm değil: **aracın kendi konumu** ve **izdüşümü**. Böylece rota, aracın bulunduğu yerden başlar — en yakın düğüme ışınlanmasını beklemes. Şeridin 1 m yanındaysan rota seni önce yumuşakça şeride çeker.

## 30 Özet: hangi ders kodun neresinde

| Öğrendiğin              | Koddaki yeri                       | Bölüm     |
|-------------------------|------------------------------------|-----------|
| dist ve prev sözlükleri | dist, prev (anahtar: kenar)        | 1         |
| Öncelik kuyruğu         | heapq.heappush/heappop             | 3         |
| Tembel silme            | if c > dist.get(...):<br>continue  | 3         |
| Hedefte durma (pop'ta!) | if v == goal_vid: break            | Soru 3    |
| Gevşetme                | if nc < dist.get((v,w),<br>inf)    | 1         |
| Negatif yasağı          | prefer_zones çarpanı, f > 0        | Soru 6, 8 |
| Yönlü graf              | directed=True, ters kenar eklenmez | Soru 2    |
| Kenar-durumlu arama     | dist[(u,v)]                        | Soru 10   |
| Bloklar                 | _is_blocked, TTL                   | Soru 9    |
| Yolu geri kurma         | chain geri sarma                   | 3         |

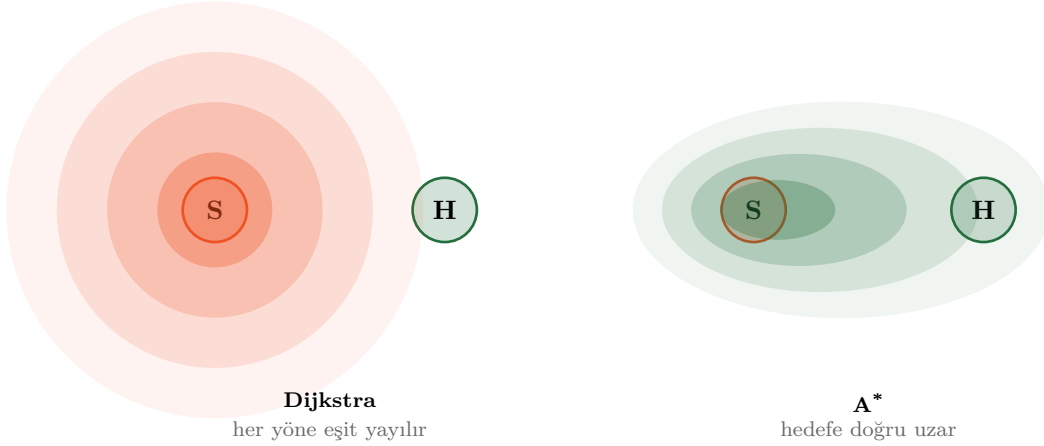
### SAVUNMA SORUSU

Bu tabloyu kapat ve şu üç soruyu sesli cevapla: (1) Neden **dist**'in anahtarı bir demet? (2) **continue** satırı kaldırılırsa ne bozulur? (3) **prefer\_zones** çarpanını  $-1$  yaparsak ne olur, kod hata verir mi?

## 6. BÖLÜM — Sonrası

### 31 A\*: Dijkstra + sezgi

Dijkstra hedefin *nerede olduğunu umursamaz* — başlangıcın etrafına her yöne eşit yayılır. Hedef doğudaysa, batıya doğru harcanan iş boşadır.



A\* tek bir değişiklik yapar: kuyruğa maliyeti değil, **maliyet + hedefe kalan tahmin** koyar.

$$f(v) = \underbrace{g(v)}_{\text{buraya kadarki gerçek maliyet}} + \underbrace{h(v)}_{\text{hedefe kalanın tahmini}}$$

Yol grafında  $h$  genelde kuş uçuşu mesafedir. Tahmin **asla gerçeği aşmazsa** (kabul edilebilir sezgi), A\* da Dijkstra kadar doğrudur — ama çok daha az düğüm açar.

$h(v) = 0$  koyarsan A\* *aynen* Dijkstra olur. Yani Dijkstra, A\*'ın “hiçbir fikrim yok” hâlidir. Haftaya bunu ve Hybrid A\*'ı (park manevrası için) işleyeceğiz.

### 32 Tuzaklar — tek sayfa özet

#### TUZAK

1. **Negatif ağırlık.** Dijkstra sessizce yanlış cevap verir. Ödülü çıkarma ile değil,  $0 < f < 1$  çarpanıyla ver.
2. **Hedefi push'ta bitirmek.** Hedef ancak **heappop** ile çıktığında kesindir. **heappush** anında değil.
3. **heapq'da güncelleme aramak.** Yoktur. Yeni kayıt push et, çıkışta bayatı **continue** ile ele.
4. **Kesinleşmiş tekrar gevşetmek.** Zararsız ama israf; büyük grafta gözle görülür.
5. **Kısıtı aramadan sonra uygulamak.** “Önce rota bul, sonra yasak dönüşleri ayıkla” çalışmaz — ayıkladığında elinde rota kalmaz. Kısıt *arama içinde* olmalı.
6. **Durumda eksik bilgi.** Kısıt “nereden geldiğine” bağlıysa durum düğüm olamaz. Bu, Soru 10'un dersi ve BURST'ün tasarımı.
7. **Süresiz blok.** Engel kalkınca rota geri gelmez. TTL koy.

8. **Float eşitlik karşılaştırması.** `dist[v] == yeni yazma; < kullan.` Kayan noktada eşitlik nadiren tam tutar.

### 33 Görevler

#### Görev 1 — Elle çöz (kâğıt, kod yok)

1. Bölümdeki 6 düğümlü grafi kâğıda çiz ve adım tablosunu **ezberden** doldur. Sonra başlangıcı F yapıp hedefi A al, tekrar çöz. Aynı cevabı almalısın (graf yönsüz) — almazsan hata nerede?

#### Görev 2 — Kendi Dijkstra'nı yaz

Naif sürümü (yığınsız) **bakmadan** yaz. Örnek 1 grafında test et, 13 çıkmalı. Sonra `heapq`'lu sürümü yaz ve iki sürümün *aynı dist* sözlüğünü ürettiğini bir `assert` ile doğrula. Rastgele 100 grafta karşılaştır — bu bir eşdeğerlik testidir ve gerçek mühendislikte böyle yapılır.

#### Görev 3 — Kırmayı öğren

Kendi kodunda **bilerek** şu üç hatayı sırayla yap ve her birinin *hangi* girdide yanlış cevap verdiğini bul: (a) hedefi `heappush` anında bitir, (b) tembel silme satırını kaldır, (c) bir kenara negatif ağırlık ver. Her biri için yanlış cevap üreten en küçük grafi yaz.

#### Görev 4 — Gerçek harita

`maps/burst_map.json` içindeki `meta.vgraph`'ı yükle (863 düğüm, 919 kenar). Kendi Dijkstra'nla iki düğüm arasında rota bul ve uzunluğunu `MissionPlanner.route_between` çıktısıyla karşılaştır. Farklıysa sebebini bul — muhtemelen U-dönüş cezası veya dönüş sınırı.

#### Görev 5 — Kenar-durumlu sürüm

Kendi Dijkstra'nı düğüm-bazlıdan kenar-bazlıya çevir. Sonra Soru 10'daki kavşağı kur ve tabela kuralını uygula. Düğüm-bazlı sürümün “rota yok” dediği yerde kenar-bazlının 16 maliyetli rotayı bulduğunu göster. **Bu görevi bitirdiğinde `mission.py`'yi yazabilecek durumdasın.**

*Dijkstra 1956'da, bir kafede kahve içerken, kâğıt kalem olmadan 20 dakikada bulunmuş.  
Sen de yarın tahtada 20 dakikada anlatabilmelisin — hedef bu.*